

Non-Clustered Index in SQL Server with Examples

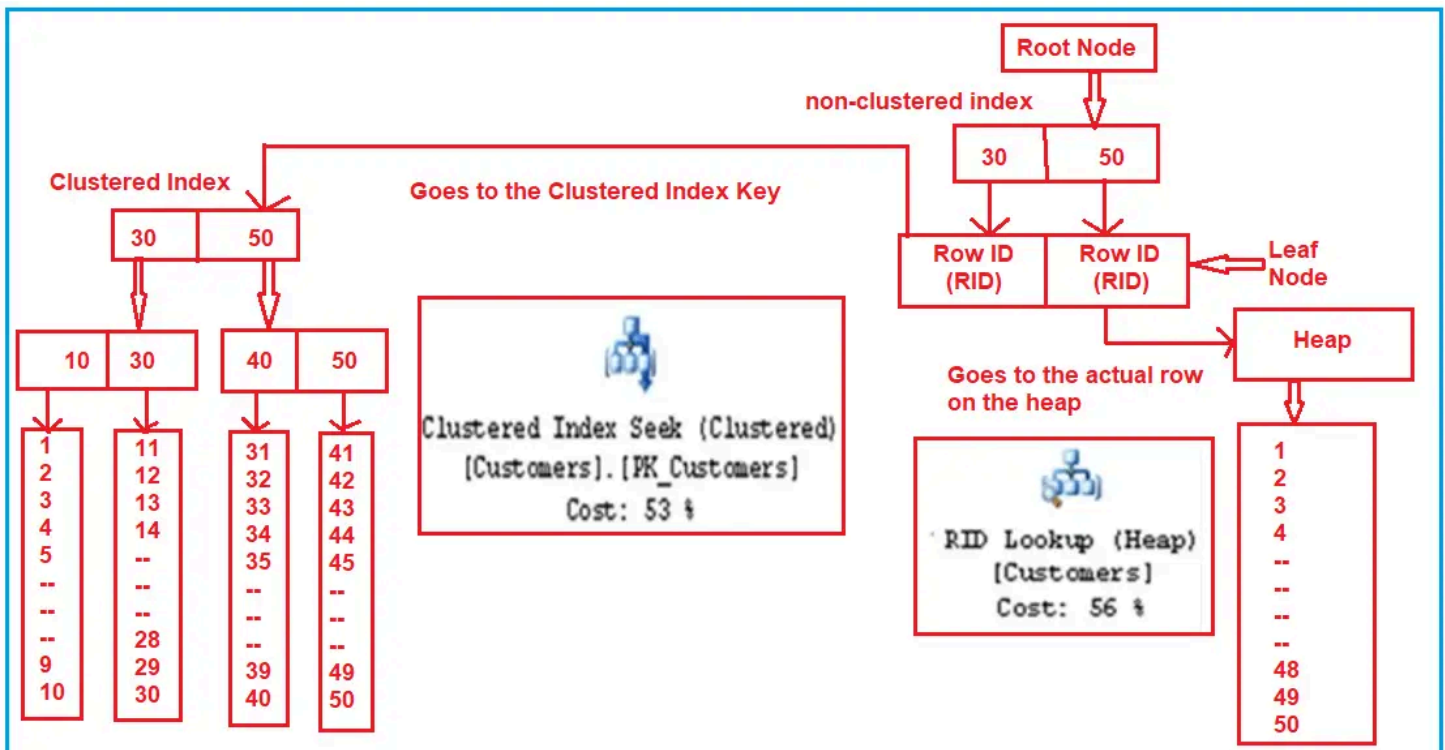
In this article, I am going to discuss the **Non-Clustered Index in SQL Server** with examples. Please read our previous article before proceeding to this article where we discussed the [Clustered Index in SQL Server](#) with examples. As part of this article, we are going to discuss the following pointers.

1. **What is the Non-Clustered Index in SQL Server?**
2. **Example to Understand the SQL Server Non-Clustered Index.**
3. **Understanding the Composite Non-Clustered Index.**
4. **What are the differences between a clustered and a non-clustered index in SQL Server?**
5. **When should we create indexes on a table?**
6. **Which Index is faster, Clustered or Non-Clustered Index?**

What is SQL Server Non-Clustered Index?

In the Non-Clustered Index, the arrangement of data in the index table will be different from the arrangement of data in the actual table. The data is stored in one place and the index table is stored in another place. Moreover, the index table will have pointers to the storage location of the actual data.

In order to understand SQL Server Non-Clustered Indexes in a better way, please have a look at the following diagram which shows the pictorial representation of the B-Tree structure of a non-clustered index in SQL Server. Both clustered and non-clustered index has the same B-Tree structure (i.e. having the Root Node, Intermediate Node (Non-Leaf Node), and Leaf Nodes). The only difference between the clustered and non-clustered index is how the leaf nodes are worked. In the case of a Clustered Index, the leaf node holds the actual data. On the other hand, the non-clustered index leaf node has a Row Identifier (RID), and this Row ID points to different things based on the situation.



Situation1:

If your table does not have a clustered index, then the Row Identifier (RID) of the non-clustered index points to the heap table. A heap table is nothing but a table without indexes. In the heap table, it will search the record row by row until it finds the data.

Situation2:

If your table has a clustered index, then the Row Identifier (RID) of the non-clustered index will point to the clustered index key and that indexed key is used to search the data.

How many non-clustered indexes can create in a table in SQL Server?

As the non-clustered indexes are created separately from the actual data, so, a table can have more than one non-clustered index in SQL Server. In the index table, the data is stored either in the ascending or descending order of the index key which does not make any effect or changes to the actual data stored in the table. In SQL Server, a maximum of 999 non-clustered indexes are created per table.

Understanding the Non-Clustered Index in SQL Server with Examples:

Let us understand the non-clustered index with an example. Please create the following tblOrder table by executing the below SQL Script. Please note that we don't have any indexes at the moment on the table.

```
CREATE TABLE tblOrder
(
    Id INT,
    CustomerId INT,
    ProductId Varchar(100),
    ProductName VARCHAR(50)
)
GO
```

Now insert some data into the table. In order to do this, here, we are inserting some mock data using a loop. Please execute the following query.

```
DECLARE @i int = 0
WHILE @i < 3000
BEGIN
    SET @i = @i + 1
    IF(@i < 500)
    Begin
        INSERT INTO tblOrder VALUES (@i, 1, 'Product - 10120', 'Laptop')
    END
    ELSE IF(@i < 1000)
    Begin
        INSERT INTO tblOrder VALUES (@i, 3, 'Product - 1020', 'Mobile')
    End
    Else if(@i < 1500)
    Begin
        INSERT INTO tblOrder VALUES (@i, 2, 'Product - 101', 'Desktop')
    End
    End
```

```

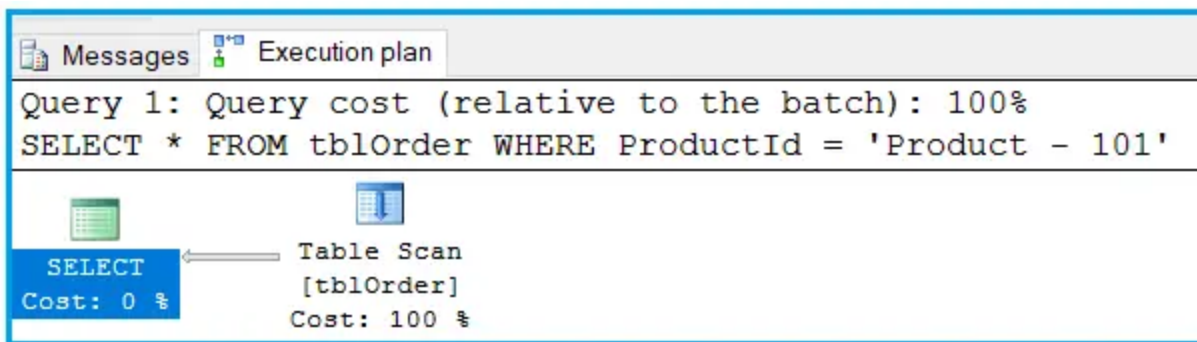
Else if(@i < 2000)
Begin
    INSERT INTO tblOrder VALUES (@i, 3, 'Product - 707', 'Pendrive')
End
Else if(@i < 2500)
Begin
    INSERT INTO tblOrder VALUES (@i, 2, 'Product - 999', 'HD')
End
Else if(@i < 3000)
Begin
    INSERT INTO tblOrder VALUES (@i, 1, 'Product - 100', 'Tablet')
End
END

```

Now, execute the following query to get the data by ProductId.

SELECT * FROM tblOrder WHERE ProductId = 'Product - 101';

After executing the above SQL Query, let us click on the Display Estimated Execution Plan button or simply click (Ctrl + L) which will open the Display Estimated Execution Plan window as shown below.



As you can see in the above image, here, the database search engine uses a table scan to get the required data.

Creating Non-clustered Index in SQL Server:

Please execute the following SQL Query which will create a non-clustered index on the ProductId column of tblOrder table.

```

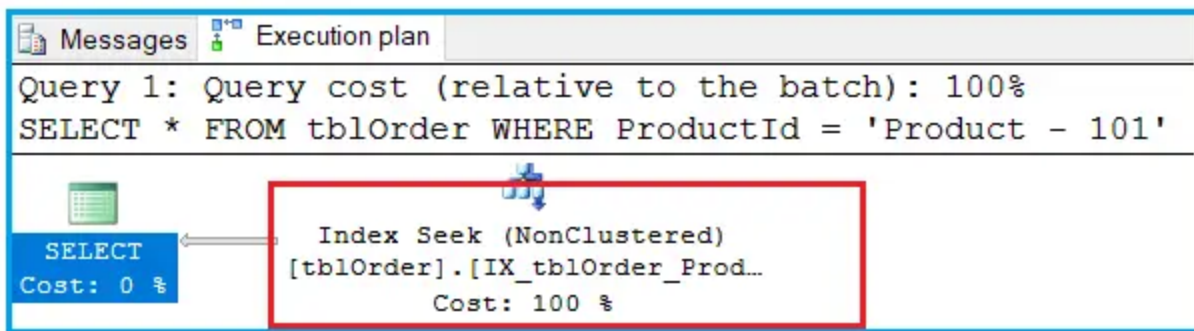
CREATE NONCLUSTERED INDEX IX_tblOrder_ProductId
ON dbo.tblOrder (ProductId)
INCLUDE ([Id],[CustomerId],[ProductName])
GO

```

Once you created the non-clustered index, now execute the following query and check the execution plan.

SELECT * FROM tblOrde WHERE ProductId = 'Product - 101';

Following is the execution plan. As you can see, it now uses a non-clustered index to get the required data.



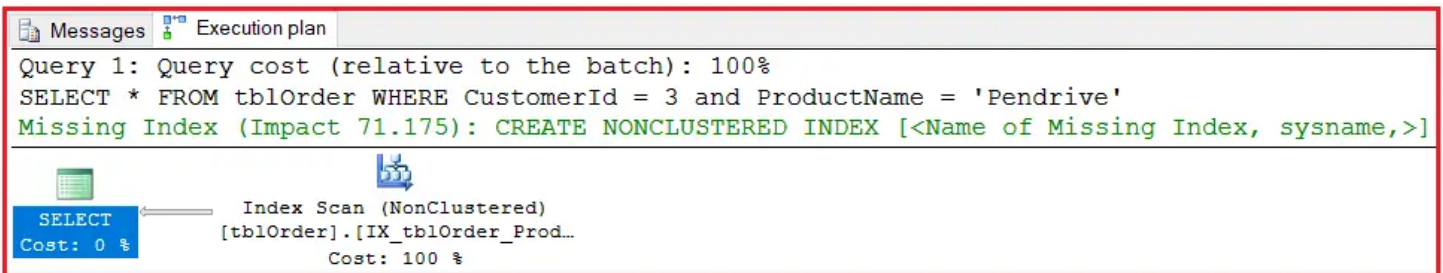
Can we create a composite Non-Clustered Index in SQL Server?

Yes. It is also possible to create a composite non-clustered index (non-clustered index based on more than one column) in SQL Server like a composite Clustered Index.

Sometimes, the **Display Estimated Execution Plan** gives you the information for creating the missing indexes which can improve the performance of the search query. Let us understand this with an example. We want to fetch the details from the order table where CustomerId = 3 and ProductName = 'Pendrive' by executing the following query.

SELECT * FROM tblOrder WHERE CustomerId = 3 and ProductName = 'Pendrive';

Once, you execute the above query, now open the Display Estimated Execution Plan window as shown below.



Now, right-click on the window and select the **Missing Index Details** option which will give you the information to create the missing indexes. In our example, it will give us information to create a composite non-clustered index based on CustomerId and ProductName. So, let us create one non-clustered index based on the CustomerId in Ascending order and ProductName in Descending Order by executing the following query.

```
CREATE NONCLUSTERED INDEX IX_tblOrder_CustomerId_ProductName
ON tblOrder(CustomerId ASC, ProductName DESC)
INCLUDE ([Id],[ProductId]);
```

Once you create the above composite non-clustered index, now execute the below query and see the execution plan which should use the composite non-clustered index to fetch the data.

SELECT * FROM tblOrder WHERE CustomerId = 3 and ProductName = 'Pendrive';

What is the difference between clustered and non-clustered indexes in SQL Server?

1. We can create only one clustered index per table whereas we can create more than one non-clustered index per table in SQL Server.
2. In the clustered index, the leaf node actually holds the data and in the case of a non-clustered index, the leaf node actually points to the leaf node of a clustered index or points to the heap table if the table does not have any clustered index.
3. The SQL Server Clustered Index determines the order in which the data is physically stored in a table and hence does not require additional disk space whereas a Non-Clustered Index in SQL Server is stored separately from the actual table, so additional disk space is required.

When should we create indexes on a table in SQL Server?

We need to create indexes on table columns provided those columns are frequently used where conditions or order by clause in a select query. It is not advised to create an index on each and every column because a number of indexes can degrade the performance of the database also. This is because every modification we make to the data should be reflected in all the index tables. [How the index impacts the performance of DML operations](#) will discuss in our next article.

Which Index is faster, Clustered or Non-Clustered Index?

The Clustered Index is slightly faster than the Non-Clustered Index in SQL Server. This is because, in the case of the clustered index, the leaf node actually holds the actual data, and hence when we search for any data, it directly gets the data from the leaf node. On the other hand, in the case of a Non-Clustered Index, the leaf node actually points to the leaf node of clustered index or to the heap table and hence there is an extra look-up from the Non-Clustered Index to the actual table (leaf node of a clustered index or heap table) to fetch the actual data.

In the next article, I am going to discuss [How Index Impacts DML Operations in SQL Server](#) with examples. Here, in this article, I try to explain **SQL Server Non-Clustered Index** with Examples. I hope you enjoy this SQL Server Non-Clustered Index with Examples article and understand how Non-Clustered Index works in SQL Server with Examples.